

Article

Reliability of LLM Inference Engines from a Static Perspective: Root Cause Analysis and Repair Suggestion via Natural Language Reports

Hongwei Li  and Yongjun Wang * 

College of Computer Science and Technology, National University of Defense Technology,
Changsha 410000, China; lhw@nudt.edu.cn

* Correspondence: wangyongjun@nudt.edu.cn

Abstract

Large Language Model (LLM) inference engines are becoming critical system infrastructure, yet their increasing architectural complexity makes defects difficult to be diagnosed and repaired. Existing reliability studies predominantly focus on model behavior or training frameworks, leaving inference engine bugs underexplored, especially in settings where execution-based debugging is impractical. We present a static, issue-centric approach for automated root cause analysis and repair suggestion generation for LLM inference engines. Based solely on issue reports and developer discussions, we construct a real-world defect dataset and annotate each issue with a semantic root cause category and affected system module. Leveraging text-based representations, our framework performs root cause classification and coarse-grained module localization without requiring code execution or specialized runtime environments. We further integrate structured repair patterns with a large language model to generate interpretable and actionable repair suggestions. Experiments on real-world issues concerning vLLMs demonstrate that our approach achieves effective root cause identification and module localization under limited and imbalanced data. A cross-engine evaluation further shows promising generalization to TensorRT-LLM. Human evaluation confirms that the generated repair suggestions are correct, useful, and clearly expressed. Our results indicate that static, issue-level analysis is a viable foundation for scalable debugging assistance in LLM inference engines. This work highlights the feasibility of static, issue-level defect analysis for complex LLM inference engines and explores automated debugging assistance techniques. The dataset and implementation will be publicly released to facilitate future research.

Keywords: LLM inference engines; root cause analysis; static issue-based diagnosis; automated repair suggestion



Academic Editors: Yan Qiao and
Francesco Cauteruccio

Received: 12 January 2026

Revised: 7 February 2026

Accepted: 12 February 2026

Published: 13 February 2026

Copyright: © 2026 by the authors.

Licensee MDPI, Basel, Switzerland.

This article is an open access article distributed under the terms and conditions of the [Creative Commons Attribution \(CC BY\)](https://creativecommons.org/licenses/by/4.0/) license.

1. Introduction

LLMs have demonstrated remarkable capabilities in natural language understanding, code generation, and multimodal reasoning, and are increasingly deployed in production systems such as search engines, intelligent assistants, and developer tools. As LLMs continue to scale in model size and deployment scope, the efficiency and reliability of the inference phase have become critical bottlenecks for real-world adoption. To address these challenges, a variety of high-performance inference engines, such as vLLM [1], TensorRT-LLM, and FasterTransformer, have been developed to optimize inference throughput and

latency through sophisticated scheduling mechanisms, memory reuse strategies, and parallel execution frameworks [2].

Compared to traditional software systems, LLM inference engines exhibit significantly higher architectural and operational complexity. Their execution pipelines typically involve dynamic request batching, asynchronous scheduling, heterogeneous hardware coordination, and fine-grained GPU memory management across multiple modules. While these optimizations substantially improve performance, they also introduce numerous failure modes that are difficult to anticipate and debug. In practice, defects in inference engines may lead to service instability, resource leaks, silent correctness issues, or even security vulnerabilities, posing serious risks to large-scale deployment. Consequently, improving the reliability of LLM inference engines has emerged as an important yet challenging research problem.

Most existing research on LLM reliability concentrates on model-level concerns, such as robustness against adversarial inputs, bias and fairness analysis, or training-time defect detection. Some recent studies have explored bug detection in deep learning frameworks and GPU computing libraries, using static analysis, dynamic tracing, or fuzzing-based techniques to uncover low-level operator bugs or concurrency issues [3–5]. However, these approaches are typically designed for general-purpose deep learning frameworks and often assume access to executable code, specialized runtime environments, or extensive dynamic traces. Such assumptions are frequently violated in the context of modern LLM inference engines, which evolve rapidly, depend on specific hardware configurations, and are costly or infeasible to reproduce in controlled environments.

In parallel, the software engineering community has developed a rich body of work on defect localization and automated program repair for conventional software systems. Techniques such as log-based root cause analysis, predicate-based debugging, and template-driven patch generation have demonstrated effectiveness in traditional settings [6–8]. AURORA [9], for example, performs predicate-based root cause analysis by correlating program state predicates with crash occurrences obtained from extensive fuzzing. Despite their success, these approaches often incur substantial computational overhead and rely heavily on dynamic execution and instrumentation [10,11]. Moreover, because inference engines exhibit a hybrid nature that combines characteristics of systems software and machine learning infrastructure, traditional defect analysis methods struggle to capture the semantic patterns underlying their bugs. As a result, their applicability to LLM inference engines remains limited [12,13].

An additional obstacle lies in the scarcity of systematically curated datasets of real-world inference engine defects. Unlike mature software domains, inference engines lack publicly available, well-annotated defect corpora that support reproducible evaluation and comparative studies. This data gap further hampers the development and validation of automated analysis and repair techniques tailored to inference engines [14].

In summary, existing research on inference engine reliability faces three key challenges: (1) the absence of publicly available datasets capturing real-world defects in LLM inference engines; (2) the heavy reliance of existing defect analysis techniques on runtime execution and dynamic information, which limits their practicality for complex and hardware-dependent systems; and (3) a lack of automated repair approaches capable of providing interpretable and actionable guidance to developers working on inference engine codebases.

To address these challenges, this paper proposes a lightweight and static approach for root cause analysis and repair suggestion generation based solely on natural language issue reports. Our method takes real-world GitHub issues (data snapshot taken in January 2026) as input and leverages defect descriptions, stack traces, and discussion contexts, without re-

quiring execution of the inference engine or access to specialized runtime environments. Specifically, we first construct and manually annotate a dataset of inference engine defects with root cause categories and affected system modules. We then design a text-based classification model to infer defect root causes and employ semantic matching techniques to localize affected modules at a coarse-grained level. Finally, we integrate predefined root cause–repair patterns with the reasoning and generation capabilities of large language models to automatically produce explanatory and actionable repair suggestions. Here, a *root cause–repair pattern* refers to an abstract, reusable mapping between a defect root cause category and a high-level repair strategy, distilled from recurring fixes observed in historical issue discussions. Each pattern describes (i) the typical failure mechanism associated with a root cause and (ii) the corresponding class of corrective actions, such as adding validation, enforcing state constraints, or introducing synchronization safeguards. These patterns serve as structured repair priors that constrain the space of possible fixes and provide semantically meaningful guidance to the language model during generation.

Experimental results demonstrate that, even with limited and imbalanced training data, the proposed approach achieves strong accuracy in both root-cause classification and module localization. Furthermore, human evaluation indicates that the generated repair suggestions are generally correct, useful, and clearly articulated, demonstrating the practical potential of our framework for assisting developers in debugging LLM inference engines and laying a foundation for future research on automated root cause analysis in LLM inference engines.

The main contributions of this paper are summarized as follows:

- We construct and release a real-world defect dataset for LLM inference engines, with systematic annotations of root cause categories and affected modules.
- We propose a static, issue-based framework for automated root cause classification and module-level fault localization without requiring execution or runtime instrumentation.
- We design a repair suggestion generation method that combines root cause–repair patterns with large language models to produce interpretable and actionable debugging guidance.
- We conduct extensive experiments and human evaluations to demonstrate the effectiveness and research merit of the proposed approach.
- We will publicly release our implementation code and dataset to foster further research in this area.

2. Background

LLM inference engines serve as critical foundational software that bridges trained models and real-world applications. Their primary responsibility is to execute the model’s forward inference process efficiently while ensuring correctness. Compared to the training phase, inference places greater emphasis on low latency, high throughput, and efficient resource utilization. Consequently, inference engines commonly integrate a wide range of system-level optimizations, such as dynamic batching, request scheduling, GPU memory reuse, and operator fusion. Through these mechanisms, inference engines can support large-scale concurrent requests under constrained hardware resources, thereby meeting the stringent performance and stability requirements of online services [1].

In contrast to their performance-oriented design goals, LLM inference engines are typically highly complex in their engineering implementation. Using representative open-source inference engines as examples, their internal architectures involve not only intricate scheduling and state management logic but also coordinated execution across model computation, memory management, and heterogeneous hardware components [15]. This multi-layered and tightly coupled structure leads to a diverse spectrum of defect types,

including configuration errors, lifecycle management bugs, concurrency-related state inconsistencies, and low-level operator faults. These defects often arise from interactions across software layers (e.g., scheduler–memory manager coupling) or across hardware boundaries (e.g., CPU–GPU coordination), which distinguishes inference engine failures from conventional application-level bugs [16]. Such defects are often difficult to detect in advance using simple unit testing or traditional static analysis techniques [3,4].

Empirical evidence from real-world development suggests that defects in LLM inference engines are numerous, difficult to localize, and costly to fix [14]. Developers typically rely on GitHub issues, log messages, and user feedback to diagnose problems. However, defect descriptions are usually expressed as unstructured natural language texts containing context-dependent information [17,18]. While such information can assist manual debugging, it is challenging for automated methods to leverage directly in the absence of systematic analysis techniques.

Issue reports in inference engine repositories typically contain heterogeneous diagnostic signals, including textual problem descriptions, stack traces, configuration snippets, hardware environment details, and developer discussions. These elements collectively provide indirect evidence about failure symptoms, triggering conditions, and potential root causes. Unlike execution traces or formal specifications, such information reflects practical debugging knowledge contributed by developers and users [19,20]. Despite its practical value, relatively few existing studies have systematically explored how to mine and utilize such rich issue data to support automated failure diagnosis and root cause analysis for LLM inference engines.

3. Design

This section presents the design of our automated framework for root cause analysis and repair suggestion generation targeting defects in LLM inference engines. The framework takes real-world issue reports as input and performs a sequence of static, text-based analyses to infer defect causes, localize affected modules, and generate actionable repair guidance. Figure 1 illustrates the overall architecture of the proposed system.

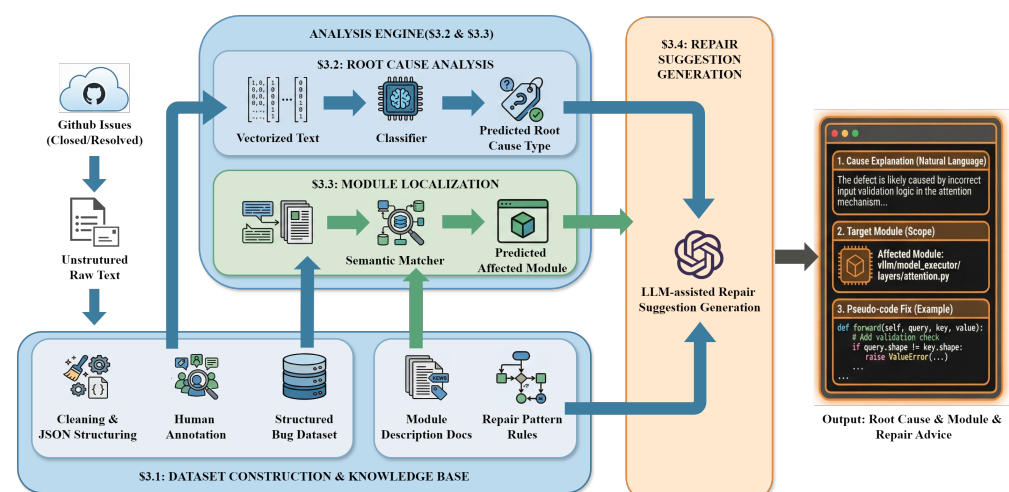


Figure 1. Overview of the proposed static analysis and repair suggestion framework for LLM inference engine defects.

Our design is guided by three core principles:

(1) Static-first and execution-free. The framework relies exclusively on information available in issue reports, such as natural language descriptions, stack traces, and discussion threads, and does not require access to the inference engine’s runtime environment,

execution traces, or specialized hardware. This makes the approach particularly suitable for bugs that are difficult or costly to reproduce.

(2) Modular and extensible pipeline. Each stage of the analysis pipeline, including root cause classification, module localization, and repair suggestion generation, is implemented as an independent component. This modularity enables individual components to be replaced or enhanced without affecting the overall framework.

(3) Interpretability-oriented design. Instead of producing opaque predictions, the framework aims to provide intermediate, interpretable outputs (root cause categories and affected modules) that naturally support downstream repair reasoning and human understanding.

3.1. Dataset Construction

To enable systematic evaluation and reproducible experimentation, we construct a defect dataset specifically targeting LLM inference engines. In this work, we focus on vLLM, a widely used open-source inference engine, and collect issue reports from its GitHub repository.

Only closed issues labeled as bugs are selected, ensuring that each defect has been acknowledged and resolved by developers. This design choice guarantees that the ground-truth root causes and fixes are available, which is critical for reliable annotation and evaluation. Each issue therefore represents a validated real-world engineering problem rather than a speculative or incomplete report.

For each selected issue, we extract multiple information sources relevant to defect diagnosis, including the issue title, detailed problem description, developer and maintainer discussions, error messages or stack traces, and any explicitly referenced source files or modules. Although these elements are primarily unstructured natural language, together they provide complementary views of the defect's manifestation, context, and resolution process.

All extracted information is normalized and stored in structured JSON format, with each issue represented as an independent data instance. We then manually annotate each issue along two dimensions:

- **Root cause category**, which captures the semantic nature of the underlying failure mechanism.
- **Affected module**, which identifies the primary subsystem of the inference engine involved in the defect.

Unlike prior bug taxonomies that enumerate fine-grained implementation-specific error types, we adopt a compact, semantically grounded root cause classification scheme. The goal is to capture why an inference engine fails at a conceptual level.

Specifically, all issues are categorized into four high-level root cause classes, as summarized in Table 1. These categories are designed to be mutually exclusive, interpretable, and applicable across different inference engines.

This abstraction deliberately decouples root cause semantics from specific implementation details, enabling more robust learning under limited data and improving cross-engine transferability.

Annotations are performed by carefully reviewing developer discussions and associated fix commits to ensure alignment with the actual resolution logic. The dataset construction procedure is summarized in Algorithm 1. Beyond supporting the experiments in this paper, the dataset can also serve as a high-fidelity exploratory dataset, designed to facilitate preliminary research into the analysis of reasoning engine deficiencies.

Table 1. Root Cause Taxonomy for LLM Inference Engine Bugs.

Root Cause	Semantic Focus	Typical Indicators
model_execution	Operator-level or numerical computation errors	kernel execution failures, incorrect outputs, numerical instability, precision loss, FP16 overflow, attention kernel errors.
runtime_state	Inference-time state management and decoding lifecycle	KV cache inconsistency, cache miss, session reset, token mismatch, streaming or decoding state errors.
resource_concurrency	Resource exhaustion and concurrent execution issues	out-of-memory (OOM), CUDA errors, NCCL communication failures, deadlocks, race conditions, multi-GPU synchronization.
io_semantic	Input/output semantics and parameter interpretation	tokenizer mismatches, special token handling (BOS/EOS), position or RoPE errors, prompt formatting, temperature or decoding parameter bugs.

Algorithm 1 Inference Engine Issue Dataset Construction**Require:** GitHub issue repository $\mathcal{R}$ **Ensure:** Labeled issue dataset $\mathcal{D}$

- 1: Initialize empty dataset $\mathcal{D}$
- 2: **for** each closed issue $I \in \mathcal{R}$ labeled as bug **do**
- 3: Extract textual fields: title, description, comments, stack traces
- 4: Extract referenced files or module names from text
- 5: Manually assign root cause label r
- 6: Manually assign affected module label m
- 7: Add (I, r, m) to dataset $\mathcal{D}$
- 8: **end for**
- 9: **return** $\mathcal{D}$

3.2. Root Cause Analysis

Given the constructed dataset, we formulate root cause analysis as a supervised text classification problem. The goal is to automatically infer the root cause category of a defect based solely on the textual content of its issue report.

Problem Formulation. Let x denote the textual representation of an issue, formed by concatenating its title, description, discussion comments, and referenced filenames. Let $r \in \mathcal{R}$ denote the corresponding root cause category from a predefined label set. The objective is to learn a classifier C such that:

$$\hat{r} = C(x) \quad (1)$$

where $\hat{r}$ is the predicted root cause.

Unlike traditional debugging techniques that depend on code execution or runtime traces, our approach exploits the observation that issue reports often contain implicit diagnostic signals. Developers frequently describe symptoms, hypothesize causes, or reference relevant subsystems in natural language, making textual analysis a viable proxy for deeper program semantics.

To preserve contextual richness, we aggregate all textual elements of an issue into a single document. This document is transformed into a numerical representation using lightweight text encoding techniques, enabling efficient training even under limited data conditions. A multi-class classifier is then trained to map the encoded text to root cause categories.

The predicted root cause serves as a high-level semantic abstraction of the defect and provides essential prior knowledge for subsequent module localization and repair reasoning.

Algorithm 2 outlines the root cause analysis procedure.

Algorithm 2 Root Cause Analysis

Require: Issue text x , trained classifier C

Ensure: Predicted root cause $\hat{r}$

- 1: Encode issue text into vector representation $v \leftarrow \text{EncodeText}(x)$
 - 2: Predict root cause $\hat{r} \leftarrow C(v)$
 - 3: **return** $\hat{r}$
-

3.3. Module Localization

After identifying the defect's root cause, the framework further localizes the fault to a specific inference engine module. Rather than attempting fine-grained localization at the function or line level, we intentionally target module-level localization to balance precision with robustness and generality.

Text-Based Module Retrieval Formulation. Let $\mathcal{M} = \{m_1, m_2, \dots, m_k\}$ denote the set of inference engine modules. Rather than relying on manually written or externally provided module documentation, each module m is represented by aggregating historical issue texts that have been manually annotated as belonging to that module. Specifically, all issue descriptions associated with the same module are concatenated to form a module-level textual profile. Given an issue text x , we compute its textual similarity with each module representation and select the most relevant module.

Formally, the predicted module is given by:

$$\hat{m} = \arg \max_{m \in \mathcal{M}} \text{Sim}(\text{EncodeText}(x), \text{EncodeText}(m)) \quad (2)$$

Here, both issue texts and module representations are encoded using the same TF-IDF vectorizer trained over the issue corpus, such that similarity reflects lexical overlap and recurring defect-related terminology observed in historical issues.

This design avoids dependence on static code structures or call graphs, which are often unavailable or unstable across engine versions. Instead of assuming semantic equivalence between heterogeneous text sources, our approach adopts a heuristic, data-driven retrieval strategy: module localization is guided by distributional similarity between an incoming issue and historical issues associated with each module. While coarse-grained, this formulation provides a lightweight and execution-free mechanism for narrowing the fault search space.

Algorithm 3 summarizes the module localization procedure.

Algorithm 3 Module Localization

Require: Issue text x , module set $\mathcal{M}$

Ensure: Predicted module $\hat{m}$

- 1: Encode issue text into vector $v \leftarrow \text{EncodeText}(x)$
 - 2: **for** each module $m \in \mathcal{M}$ **do**
 - 3: Encode aggregated module text $u_m \leftarrow \text{EncodeText}(\text{Desc}(m))$
 - 4: Compute similarity score $s_m \leftarrow \text{Sim}(v, u_m)$
 - 5: **end for**
 - 6: $\hat{m} \leftarrow \arg \max_m s_m$
 - 7: **return** $\hat{m}$
-

3.4. Repair Suggestion Generation

The final stage of the framework aims to generate repair suggestions based on the inferred root cause and the localized module. Rather than directly synthesizing deployable patches, which would require precise program analysis and formal verification, we focus on producing natural-language repair guidance supplemented with illustrative pseudocode to assist developers in debugging and fixing defects.

Repair Pattern Abstraction. Through manual analysis of previously resolved issues, we observe that many defects exhibit recurring repair strategies within the same root cause category. For instance, configuration-related defects often require stricter validation or sanity checks, whereas engine state issues typically involve enforcing lifecycle constraints or introducing defensive assertions.

We abstract these recurring strategies into a set of repair patterns indexed by root cause type. Given a predicted root cause $\hat{r}$ and a localized module $\hat{m}$, the system selects the corresponding repair pattern and instantiates it with issue-specific contextual information.

LLM-Assisted Repair Generation. To improve the fluency, clarity, and contextual relevance of the repair suggestions, we employ a LLM in the final generation step. The LLM is guided by a structured and constrained prompt that incorporates the original issue text, the predicted root cause, the affected module, and the selected repair pattern. This design limits the generation space and helps mitigate hallucinated or overly generic outputs.

Each generated repair suggestion consists of three components: (1) an explanation of the underlying defect cause, (2) identification of the module or component that should be modified, and (3) concrete repair actions expressed in natural language, optionally accompanied by pseudocode.

Algorithm 4 outlines the overall repair suggestion generation process.

Structured Prompt Design. To ensure that the LLM produces focused, actionable, and non-hallucinatory repair guidance, we adopt a carefully structured prompt design. Instead of relying on free-form text generation, the prompt explicitly encodes the intermediate analysis results produced by earlier stages of the framework, including the predicted root cause category and the localized module.

The prompt is designed with three objectives: (1) grounding the LLM's reasoning in the concrete issue context; (2) constraining generation to the inferred defect cause and scope; and (3) encouraging structured, developer-oriented repair guidance rather than speculative explanations.

Specifically, given an issue text x , predicted root cause $\hat{r}$, and affected module $\hat{m}$, we construct the following prompt template:

Context: You are analyzing a bug report from an LLM inference engine.

Issue Description: [Issue text x]

Identified Root Cause Category: [$\hat{r}$]

Affected Module: [$\hat{m}$]

Task: Based on the issue description and the identified root cause, explain the likely reason for the bug. Then, provide concrete and actionable repair suggestions for the affected module. If appropriate, include brief pseudocode or code-level guidance to illustrate the fix. Focus on practical debugging and repair steps rather than abstract explanations.

This structured prompt explicitly anchors the LLM's generation to the framework's intermediate outputs, thereby reducing the space of plausible responses. In practice, this approach significantly improves the relevance and consistency of generated repair suggestions while mitigating hallucinated or overly generic outputs.

Finally, we emphasize that the LLM is used solely as a conditional text generator in the final stage of the pipeline. All diagnostic reasoning, such as root cause classification and

module localization, is performed independently by deterministic or statistical components, preserving the interpretability and reproducibility of the overall framework.

Algorithm 4 Repair Suggestion Generation

Require: Issue text x , predicted root cause $\hat{r}$, localized module $\hat{m}$

Ensure: Repair suggestion S

- 1: Construct structured prompt $p \leftarrow \text{BuildPrompt}(x, \hat{r}, \hat{m})$
 - 2: Generate repair suggestion $S \leftarrow \text{LLMGenerate}(p)$
 - 3: **return** S
-

4. Implementation

This section describes the concrete implementation of the proposed automated framework for bug analysis and repair suggestion generation. Each component is implemented in alignment with the design principles introduced in Section 3, with an emphasis on reproducibility, efficiency, and practical deployability.

4.1. Data Processing and Text Construction

For each GitHub issue, we construct a unified textual representation by concatenating multiple information sources, including the issue title, detailed problem description, selected discussion comments, error logs, stack traces, and referenced source code file names. This consolidated representation serves as the common input for both root cause classification and module localization.

During preprocessing, all text is normalized by converting characters to lowercase and removing redundant whitespace. For stack traces and code snippets, we apply lightweight normalization rules: memory addresses, line numbers, and non-informative tokens are removed, while function names, file paths, and error-related keywords are preserved. This strategy reduces textual noise while retaining semantic cues that are critical for diagnosing inference engine defects.

The resulting text representation balances completeness and conciseness, ensuring that both high-level symptom descriptions and low-level technical hints are available to downstream analysis components.

4.2. Root Cause Analysis Implementation

The root cause classification module corresponds to the classifier C defined in Algorithm 2. It is implemented as a supervised text classification pipeline.

Text Encoding (EncodeText). The `EncodeText` function is implemented using term frequency–inverse document frequency (TF-IDF) vectorization. We adopt the TF-IDF implementation from the scikit-learn library to convert issue texts into sparse numerical vectors. Unigrams and bigrams are used to capture both individual diagnostic keywords and short semantic phrases. To mitigate overfitting on the small dataset, low-frequency terms are filtered, and standard English stop words are removed.

Classification Model (C). We implement the classifier C as a multinomial logistic regression model with class-weight balancing. This choice is motivated by several considerations: (1) logistic regression is computationally efficient and stable under limited data; (2) its linear decision boundary provides interpretability in terms of feature importance; and (3) it exhibits strong robustness compared to more complex nonlinear models under class imbalance.

Model training and evaluation are performed using stratified k -fold cross-validation to ensure fair performance estimation across imbalanced root cause categories.

4.3. Module Localization Implementation

The module localization component follows Algorithm 3 and is likewise implemented using text-based representations and similarity computation.

Module Description Construction. For each inference engine module, we manually construct a short descriptive document that includes the module name, its primary responsibilities, and commonly associated file paths or keywords. These descriptions serve as semantic prototypes for each module.

Similarity Computation. Both issue texts and module descriptions are encoded using the same TF-IDF vectorization scheme employed in root cause classification, ensuring that all representations reside in a shared vector space. For each issue, we compute the cosine similarity between its vector representation and each module vector. The module with the highest similarity score is selected as the predicted affected module.

This lightweight similarity-based approach enables efficient module-level localization without requiring access to the inference engine's source code structure or runtime behavior.

4.4. Repair Suggestion Generation Implementation

The repair suggestion generation component implements Algorithm 4 and relies on conditional natural-language generation using a large language model.

Prompt Construction (BuildPrompt). We implement a structured prompt template that integrates the original issue description with the predicted root cause category and localized module. The prompt explicitly instructs the LLM to (1) explain the likely cause of the defect, and (2) provide concrete, actionable repair suggestions tailored to the affected module. This design constrains the generation space and encourages focused, developer-oriented outputs.

Text Generation (GenerateText). Repair suggestions are generated by invoking the text generation API of a pre-trained, off-the-shelf large language model (GPT-5.2 & Qwen3-235B-A22B). The model is used without any additional fine-tuning. The structured prompt incorporates the issue description, predicted root cause category, predicted module, and brief contextual instructions. The generated output typically includes a concise explanation of the defect mechanism, followed by recommended repair actions and, where appropriate, illustrative pseudocode.

Implementation Environment. All experiments are conducted in a standard Linux environment. Data processing, text encoding, and machine learning components are implemented in Python 3.10 using widely adopted libraries such as NumPy and scikit-learn. The repair suggestion generation module interfaces with an external LLM API, enabling flexible substitution of different language models without modifying the rest of the framework.

5. Experimental Evaluation

This section presents a comprehensive experimental evaluation of the proposed automated framework for root cause analysis and repair suggestion generation for LLM inference engines. The evaluation aims to answer the following research questions:

- **RQ1:** How accurately can the proposed approach identify the root causes of inference engine bugs?
- **RQ2:** How effective and efficient is the proposed module localization method?
- **RQ3:** Can the proposed approach generalize across different LLM inference engines?
- **RQ4:** How useful are the automatically generated repair suggestions in practice?

5.1. Experimental Setup

Dataset. The experiments are conducted on a real-world dataset of closed bug reports collected from open-source LLM inference engine projects, including vLLM and TensorRT-LLM. Each issue contains a title, detailed description, discussion comments, and relevant metadata. For evaluation, each issue is annotated with both a root cause category and an affected functional module. After filtering incomplete or low-information records, the final dataset comprises **176** vLLM issues and **100** TensorRT-LLM issues.

Table 2 summarizes the final label distribution for the vLLM dataset. As is typical in real-world bug repositories, the dataset exhibits a pronounced long-tailed distribution, with `resource_concurrency` and `model_execution` failures accounting for the majority of reported issues.

Table 2. Root Cause Distribution in vLLM Dataset.

Root Cause Category	Issue ID	Ratio (%)
<code>resource_concurrency</code>	94	53.4
<code>model_execution</code>	60	34.1
<code>io_semantic</code>	16	9.1
<code>runtime_state</code>	6	3.4
Total	176	100.0

All experiments are performed on a server running Ubuntu 22.04 with an Intel Xeon Gold 6430 CPU and 256 GB RAM. All text processing, feature extraction, and classification models are implemented in Python using standard machine learning libraries. No GPU acceleration is required for root cause classification or module localization.

For root cause classification and module localization, we report *Accuracy* and *Macro-F1*.

5.2. Root Cause Analysis Accuracy (RQ1)

We evaluate root cause classification performance using stratified 5-fold cross-validation. In each fold, 80% of the data is used for training and the remaining 20% for testing. Results are averaged across all folds.

As shown in Table 3, the proposed TF-IDF-based logistic regression model consistently outperforms all baseline methods. It achieves an absolute accuracy improvement of 18.8 percentage points over Linear SVM and 18.8 points over Random Forest.

Despite the limited dataset size and severe class imbalance, the proposed classifier attains an average accuracy of 68.8% and a Macro-F1 score of 0.421. This result is particularly notable given that classification relies exclusively on unstructured issue-level text, without access to runtime traces, execution logs, or source-level debugging information.

We further evaluate a latent semantic encoding based on TF-IDF followed by Latent Semantic Analysis (LSA), in order to assess whether a richer, lower-dimensional semantic representation improves root cause classification performance. Using the same experimental protocol and classifier (logistic regression), the TF-IDF + LSA variant achieves an average accuracy of 0.682 and a Macro-F1 score of 0.416, which is comparable to but does not consistently outperform the plain TF-IDF representation.

Table 3. Root Cause Classification Results.

Method	Accuracy	Macro-F1	Training Time (s)
TF-IDF + Linear SVM	0.500	0.266	0.06
TF-IDF + Random Forest	0.500	0.265	0.79
TF-IDF + Logistic Regression (Ours)	0.688	0.421	0.40

This result suggests that, for inference engine issue reports, explicit lexical cues—such as module names, API identifiers, error codes, and diagnostic keywords—already provide strong discriminative signals. Compressing the representation into a latent semantic space may partially dilute these fine-grained technical cues, especially under limited and imbalanced data settings. Similar observations have been reported in prior software engineering studies on bug report classification and log analysis, where sparse lexical features often outperform latent or dense semantic representations when the texts are highly technical and domain-specific.

Figure 2 presents the confusion matrix of the root cause classifier. Most misclassifications occur between the `resource_concurrency` and `model_execution` categories, reflecting the tight coupling between low-level system resource management and model execution workflows in modern LLM serving systems. Rare categories such as `runtime_state` remain challenging due to data sparsity, a limitation that mirrors real-world bug distributions.

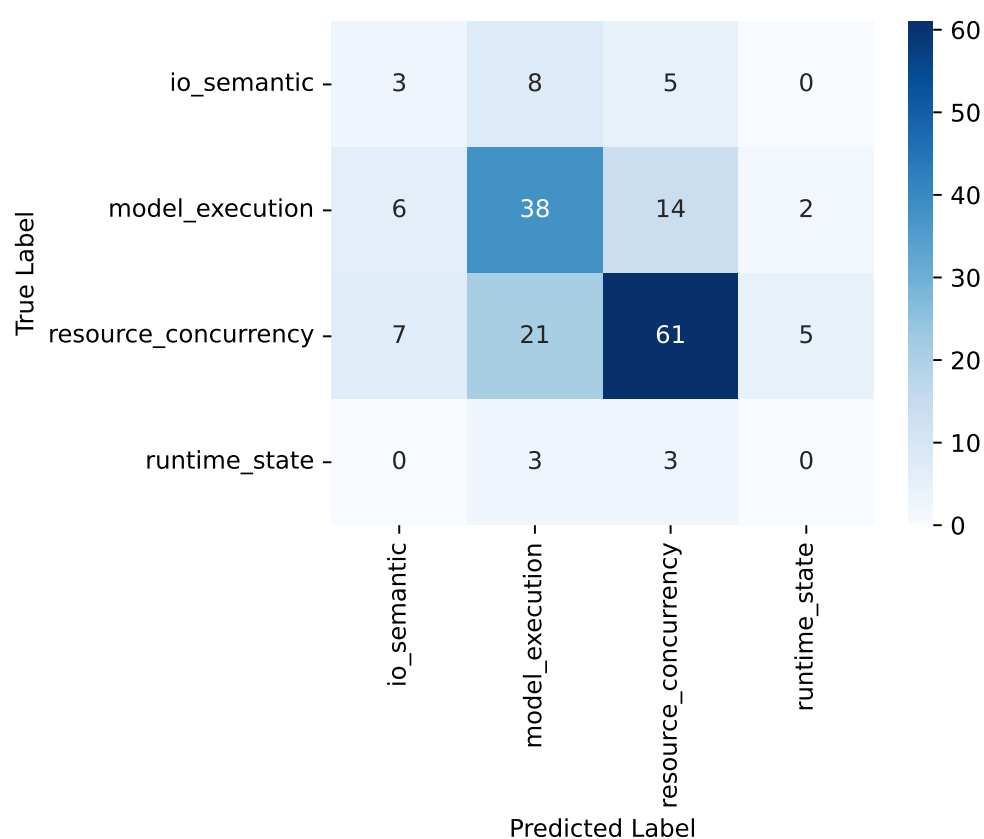


Figure 2. Confusion Matrix of Root Cause Classification.

5.3. Module Localization Effectiveness and Efficiency (RQ2)

We evaluate module localization using the proposed text-similarity-based approach. Each issue is encoded as a TF-IDF vector and compared against textual representations of inference engine modules. We report both Top-1 and Top-2 accuracy.

The results, as shown in Table 4, demonstrate that the proposed method substantially outperforms simple baseline methods. In over 84% of cases, the correct module appears within the top two predictions. This level of accuracy is practically valuable, as developers only need to focus on inspecting a subset of candidate modules rather than searching the entire codebase.

Table 4. Module Localization Results.

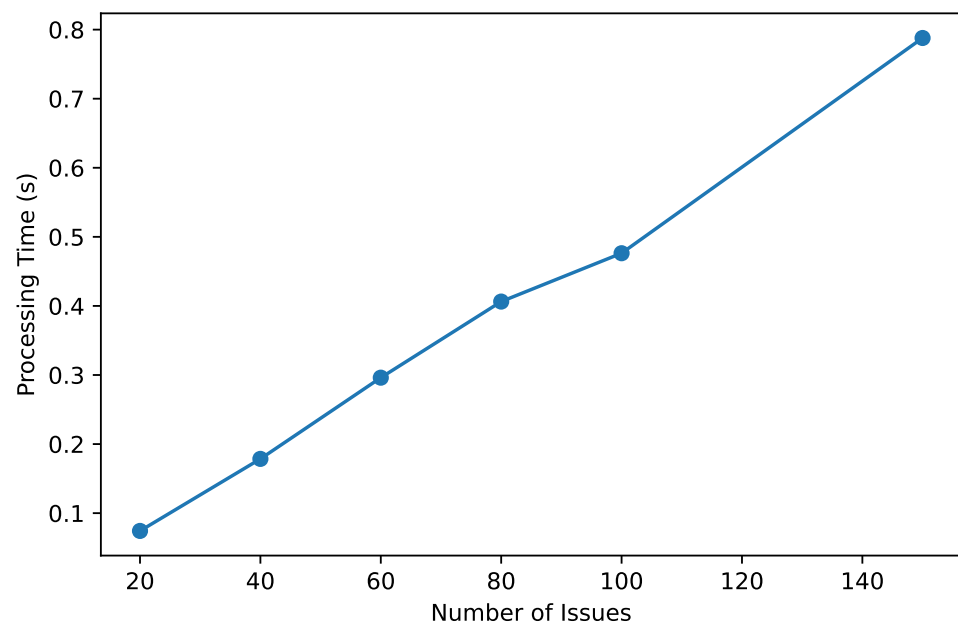
Method	Top-1 Accuracy	Top-2 Accuracy
Text Similarity (Ours)	0.705	0.841
Most Frequent Module	0.182	0.304
Random Guess	0.062	0.125

In addition to localization accuracy, we further evaluate the computational efficiency of the proposed module localization approach. Since the method is designed as a lightweight, static analysis technique intended for offline issue triaging, we focus on its scalability with respect to the number of analyzed issues.

Specifically, we measure the end-to-end processing time required for module localization under varying dataset sizes. For each dataset size, a subset of issues is randomly sampled, and the average localization time is recorded. All experiments are conducted on the same hardware and software environment, with the text representations and module descriptions fixed to eliminate confounding factors. The reported time corresponds to the average processing cost over multiple runs.

Figure 3 illustrates the relationship between the number of issues and the average processing time. The results show a near-linear growth trend, which is consistent with the theoretical time complexity of the approach. Even for a dataset containing 140 issues, the total processing time remains on the order of tens of milliseconds, indicating that the computational overhead of the method is negligible in practical applications.

These results demonstrate that the proposed module localization approach is highly efficient and scales well with dataset size, making it suitable for real-world deployment scenarios where large numbers of bug reports must be analyzed with minimal computational cost.

**Figure 3.** Scalability of Module Localization under Different Dataset Sizes.

5.4. Cross-Engine Generalization Study (RQ3)

To assess cross-engine generalization, we train the root cause classifier exclusively on vLLM issues and directly evaluate it on TensorRT-LLM issues without retraining. This setting simulates realistic deployment scenarios in which labeled data for a new inference engine is limited or unavailable.

As shown in Table 5, experimental results demonstrate that, despite differences across engines, the logistic regression model still demonstrates strong robustness, achieving an accuracy of 64.0% and a Macro-F1 score of 0.405. This suggests that simpler linear models capturing high-level lexical patterns are more transferable across heterogeneous inference engines than more complex models that may overfit engine-specific terminology.

Table 5. Cross-Engine Root Cause Classification Results.

Method	Accuracy	Macro-F1
TF-IDF + Linear SVM	0.610	0.337
TF-IDF + Random Forest	0.550	0.317
TF-IDF + Logistic Regression (Ours)	0.640	0.405

5.5. Human Evaluation of Repair Suggestions (RQ4)

We conduct a human evaluation to assess the quality of the automatically generated repair suggestions produced by our framework. The objective of this evaluation is to qualitatively examine whether the generated suggestions are technically sound, practically useful, and clearly expressed, rather than to provide statistically definitive conclusions.

Following common practice in prior studies on automated debugging and program repair, fifty issues are randomly sampled from the test set, covering different root cause categories and functional modules. For each issue, the framework generates a natural-language repair suggestion based on the predicted root cause and localized module.

Each suggestion is independently evaluated by five researchers with software engineering experience and familiarity with large-scale machine learning systems. Evaluators are provided with the original issue report, the predicted root cause and module, and the corresponding repair suggestion, while ground-truth labels are withheld to mitigate confirmation bias.

The evaluation adopts a 5-point Likert scale (1–5) for each metric, where higher scores indicate better quality. The evaluated dimensions are defined as follows:

- **Correctness:** Whether the suggestion accurately reflects the underlying cause of the issue and proposes a technically plausible fix;
- **Usefulness:** Whether the suggestion provides actionable guidance that could assist developers in diagnosing or resolving the issue;
- **Clarity:** Whether the suggestion is clearly written and easy to understand.

To assess agreement among multiple evaluators, we compute inter-rater reliability using Fleiss' κ . The resulting κ scores are 0.48 for correctness, 0.44 for usefulness, and 0.61 for clarity, indicating moderate to substantial agreement.

As shown in Table 6, the generated repair suggestions achieve consistently favorable scores across all three dimensions. The relatively high clarity scores indicate that the suggestions are easy to interpret, while the correctness and usefulness scores suggest that most suggestions align well with developers' expectations.

Table 6. Human Evaluation of Repair Suggestions Using Different LLMs.

Model	Metric	Avg. Score	Std. Dev.	Median
ChatGPT-5.2	Correctness	3.7	0.5	3.9
	Usefulness	3.6	0.6	4.0
	Clarity	4.3	0.5	4.2
Qwen3-235B-A22B	Correctness	3.5	0.6	3.6
	Usefulness	3.4	0.6	3.7
	Clarity	4.1	0.5	4.0

We further evaluate an open-source large language model (Qwen3-235B-A22B) using the same repair generation pipeline and human evaluation protocol. The results are broadly comparable to those obtained with ChatGPT, indicating that the proposed root cause-guided repair framework is largely model-agnostic and does not rely on proprietary LLM-specific behaviors.

We observe that the quality of repair suggestions is strongly correlated with the accuracy of upstream root cause classification and module localization. When both predictions are correct, evaluators report that suggestions are more specific and actionable; in contrast, errors in upstream predictions often lead to generic or partially relevant suggestions. This observation highlights the cascading effect of earlier analysis stages on the overall quality of automated repair assistance.

To contextualize the contribution of the large language model, we qualitatively compare the generated suggestions with simple templated repair patterns derived solely from root cause categories. We find that LLM-generated suggestions provide more context-aware and issue-specific guidance, often referencing concrete symptoms or components that templated patterns fail to capture, suggesting that the LLM contributes additional semantic reasoning beyond fixed heuristics.

While the evaluation involves multiple independent evaluators, the number of evaluated issues remains limited. Expanding both the evaluator pool and the number of evaluated issues represents an important direction for future work.

6. Discussion and Future Work

The automated bug analysis framework proposed in this paper is entirely based on static, issue-level information and does not require access to the runtime environment or dynamic execution of the inference engine. As a result, the approach exhibits strong adaptability across heterogeneous hardware platforms, software versions, and deployment configurations. This property is particularly valuable for LLM inference engines, where many bugs are difficult to reproduce, hardware-dependent, or tightly coupled with specific runtime states. By avoiding reliance on execution traces or online instrumentation, our method remains lightweight and practical for real-world engineering workflows.

Another key strength of the proposed approach lies in its modular pipeline design. Each stage, namely root cause classification, module localization, and repair suggestion generation, is decoupled and can be independently replaced, extended, or upgraded. This modularity enables flexible integration of alternative classifiers, more advanced localization techniques, or improved language models without redesigning the entire system. As a result, the framework provides a reusable and extensible foundation for future research on automated debugging and maintenance of inference engines.

Despite the encouraging experimental results, several limitations remain. First, both root cause classification and module localization rely primarily on textual information extracted from issue reports and discussion threads. Consequently, their effectiveness is inherently constrained by the completeness, clarity, and technical detail of the reported issues. For bug reports with vague descriptions, missing logs, or limited contextual information, the model may produce inaccurate or overly coarse predictions. This limitation reflects a broader challenge shared by many issue-driven analysis approaches in software engineering. Second, the current localization capability is restricted to coarse-grained module-level identification and does not yet narrow faults down to specific functions or source code regions. While this level of localization can substantially reduce the debugging search space and guide developers toward relevant components, additional manual investigation is still required to identify the exact fault location in practice.

Based on these observations, several promising directions for future work emerge. First, incorporating additional structured and semi-structured artifacts, such as referenced commits, code diffs, stack traces, or dependency relations mentioned in issue discussions, could further enhance the robustness and accuracy of both root cause classification and module localization. Second, extending the localization granularity from the module level to the function or code-block level would provide more precise debugging guidance and enable tighter integration with automated program repair and testing techniques. Finally, expanding the dataset to include a broader range of inference engines and defect categories would allow for more comprehensive evaluation of generalization capability and further strengthen the practical applicability of the proposed framework.

7. Conclusions

To address the heavy reliance on manual expertise in bug analysis and repair for LLM inference engines, this paper presents an automated framework that leverages issue-level textual information to perform bug diagnosis and generate repair suggestions. To enable systematic investigation, we construct a real-world bug dataset specifically focused on LLM inference engines. By decomposing the bug analysis workflow into three stages, namely root cause analysis, module localization, and repair suggestion generation, we demonstrate that lightweight text-based analysis, combined with large language models, can effectively support end-to-end bug understanding without requiring engine execution or bug reproduction.

Experimental results suggest that automatic root cause classification based on issue reports is feasible, but the achieved performance remains modest and is constrained by data scale and class imbalance. Moreover, the generated repair suggestions are found to be practically useful and interpretable, offering actionable guidance to developers throughout the debugging process.

This paper presents a feasibility and exploratory study on automated bug root cause analysis for LLM inference engines. We hope that this work provides a new perspective on improving the reliability and maintainability of LLM inference engines, and that the dataset, methodology, and insights presented here can serve as a foundation for future research and tool development. In future work, we plan to expand the dataset scale, incorporate richer program artifacts, and explore more fine-grained and fully automated bug localization and repair techniques.

Author Contributions: Conceptualization, H.L.; methodology, H.L.; software, H.L.; validation, H.L. and Y.W.; formal analysis, H.L.; investigation, H.L.; resources, H.L.; data curation, H.L.; writing—original draft preparation, H.L.; writing—review and editing, Y.W.; visualization, H.L.; supervision, H.L.; project administration, H.L. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding. The APC was funded by the authors.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The code and dataset supporting the findings of this study will be openly available in a GitHub repository upon publication at <https://github.com/moxi828/BDCC/tree/main> (accessed on 11 February 2026).

Acknowledgments: The authors thank all participants who contributed their time to complete the questionnaire. We also acknowledge the open-source community for the tools and libraries that facilitated this research.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Kwon, W.; Li, Z. vLLM: Easy, Fast, and Cheap LLM Serving with PagedAttention. 2023. Available online: <https://blog.vllm.ai/2023/06/20/vllm.html> (accessed on 11 January 2026).
2. Zhang, L.; Zhang, Z.; Li, R.; Tian, Z.; Mei, S.; Li, D. Dovetail: A CPU/GPU Heterogeneous Speculative Decoding for LLM Inference. In Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP '25), Suzhou, China, 4–9 November 2025; pp. 17393–17406.
3. Yang, Y.; He, T.; Xia, Z.; Feng, Y. A comprehensive empirical study on bug characteristics of deep learning frameworks. *Inf. Softw. Technol.* **2022**, *151*, 107004. [\[CrossRef\]](#)
4. Chen, J.; Liang, Y.; Shen, Q.; Jiang, J.; Li, S. Toward understanding deep learning framework bugs. *ACM Trans. Softw. Eng. Methodol.* **2023**, *32*, 1–31. [\[CrossRef\]](#)
5. Makkouk, T.; Wang, G.; Zhang, C.; Chen, T. An Empirical Study on Performance Bugs in Deep Learning Frameworks. In Proceedings of the 2022 IEEE International Conference on Software Maintenance and Evolution (ICSME), Limassol, Cyprus, 3–7 October 2022; pp. 35–46.
6. Liu, C.; Yang, J.; Tan, L.; Hafiz, M. R2Fix: Automatically Generating Bug Fixes from Bug Reports. In Proceedings of the 2013 IEEE Sixth International Conference on Software Testing, Verification and Validation, Luxembourg, 18–22 March 2013.
7. Mohsen, A.M.; Hassan, H.; Moawad, R.; Makady, S. A Review on Software Bug Localization Techniques using a Motivational Example. *Int. J. Adv. Comput. Sci.* **2022**, *13*, 251–261.
8. Xia, C.S.; Xing, Z.; Lo, D.; Grundy, J. Automated Program Repair in the Era of Large Pre-trained Language Models. In Proceedings of the IEEE/ACM 45th International Conference on Software Engineering (ICSE), Melbourne, Australia, 14–20 May 2023; pp. 1–13.
9. Blazytko, T.; Sykora, M.; Muench, M.; Nisi, P.; Ulz, M.; Holz, T. AURORA: Statistical Crash Analysis for Automated Root Cause Explanation. In Proceedings of the 29th USENIX Security Symposium (USENIX Security'20), Boston, MA, USA, 12–14 August 2020; pp. 235–252.
10. Li, Z.; Jiang, Z.; Huang, Q.; Gu, Q. LLM-BL: Large Language Models Are Zero-Shot Rankers for Bug Localization. In Proceedings of the 2025 IEEE/ACM 33rd International Conference on Program Comprehension (ICPC), Ottawa, ON, Canada, 27–28 April 2025.
11. Jiang, Z.; Jiang, X.; Hazimeh, A.; Tang, C.; Zhang, C.; Payer, M. Igor: Crash deduplication through root-cause clustering. In Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security, Virtual, 15–19 November 2021; pp. 3318–3336.
12. Monperrus, M. Automatic Software Repair: A Bibliography. *ACM Comput. Surv.* **2018**, *51*, 17:1–17:24. [\[CrossRef\]](#)
13. Gao, Y.; Li, Y.; Xie, X.; Wen, M.; Chen, Z. A Survey of Learning-based Automated Program Repair. *ACM Comput. Surv.* **2022**, *55*, 1–36. [\[CrossRef\]](#)
14. Liu, M.; Zhong, S.; Bi, W.; Zhang, Y.; Chen, Z.; Chen, Z.; Liu, X.; Ma, Y. A First Look at Bugs in LLM Inference Engines. *ACM Trans. Softw. Eng. Methodol.* **2026**, *accepted*.
15. Kwon, W.; Li, Z.; Zhuang, S.; Sheng, Y.; Zheng, L.; Yu, C.H.; Gonzalez, J.E.; Zhang, H.; Stoica, I. Efficient Memory Management for Large Language Model Serving with PagedAttention. In Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP '23), Koblenz, Germany, 23–26 October 2023.
16. Gao, Y.; Dou, W.; Qin, F.; Gao, C.; Wang, D.; Wei, J.; Huang, R.; Zhou, L.; Wu, Y. An Empirical Study on Crash Recovery Bugs in Large-Scale Distributed Systems. In Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, Lake Buena Vista, FL, USA, 4–9 November 2018.
17. Laiq, M.; Dobslaw, F. Automatic techniques for issue report classification: A systematic mapping study. *arXiv* **2025**, arXiv:2505.01469.
18. Motwani, M.; Brun, Y. Better Automatic Program Repair by Using Bug Reports and Tests Together. In Proceedings of the 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE), Melbourne, Australia, 14–20 May 2023; pp. 1225–1237.
19. Hirsch, T.; Hofer, B. Root Cause Prediction Based on Bug Reports. In Proceedings of the 2020 IEEE International Symposium on Software Reliability Engineering Workshops (ISSREW), Coimbra, Portugal, 12–15 October 2020.
20. Koyuncu, A.; Liu, K.; Bissyandé, T.F.; Kim, D.; Monperrus, M.; Klein, J.; Le Traon, Y. iFixR: Bug Report Driven Program Repair. In Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE'19), Tallinn, Estonia, 26–30 August 2019; pp. 403–414.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.